MediCare – Clinic Management & Appointment System

Project Documentation

Team Members: Mohamed Munir (TL), Malak Hany, Ashraf Ibrahim, Leqaa Ibrahim, Natalie Bola

1. Project Planning & Management

1.1 Project Proposal

Overview. MediCare is a web-based healthcare platform that lets patients search for doctors by specialization, book appointments, and access their medical history. Doctors manage schedules, confirm or reject requests, write prescriptions, and track patient records. Administrators oversee doctors, patients, specializations, appointments, and analytics. The system is built with ASP.NET Core MVC using an N-Tier architecture.

Problem statement. Many small and mid-sized clinics still manage appointments by phone or paper. This causes double bookings, missed appointments, lost records, and no visibility into clinic performance.

Objectives.

- Provide a single platform for booking, managing, and tracking appointments without conflicts.
- Give doctors a dashboard for schedules, patients, medical records, and prescriptions.
- Keep complete, secure digital medical records per patient.
- Send real-time (SignalR), SMS (Twilio), and email (SendGrid) notifications and reminders.
- Give admins analytics (Chart.js) and management tools.
- Deploy a live, documented system on Microsoft Azure.

Scope – In. Multi-role authentication (Patient, Doctor, Admin); doctor listing with search/filter; calendar-based booking, cancel, reschedule, conflict detection; doctor dashboard and working hours/vacations; medical records and file uploads; digital prescriptions with print format; notifications; patient profile; admin dashboard; analytics and PDF/Excel export; dark/light mode; Azure deployment with CI/CD.

Scope – Out (this release). Online payment gateway, video consultations, insurance claims, native mobile app, multi-clinic/multi-branch support, integration with external hospital systems.

Item	Details
Technology stack	ASP.NET Core MVC, Entity Framework Core, ASP.NET Identity, SignalR, Twilio, SendGrid, FullCalendar.js, Chart.js, Bootstrap 5, SQL Server / Azure SQL, Azure App Service, GitHub Actions or Azure DevOps
Design patterns	N-Tier (Web / Services / Data), Repository, Unit of Work, Factory (appointments and notifications), Dependency Injection
Expected deliverables	Working system on Azure, source code on GitHub, project documentation (architecture with diagrams, API documentation, setup guide, user manual), test report, final presentation
Assumptions	Free/student Azure credits and Twilio/SendGrid trial accounts are available; one clinic is modeled; Arabic/English UI is optional

1.2 Project Plan

The course content spans 4 weeks of features; the remaining time up to 30 Nov is buffer for integration, testing, and overruns.

Phase	Dates	Milestone
Planning & Management	2 Oct – 16 Oct 2026	M0: Proposal, plan, risks, KPIs, requirements approved
Implementation – Week 1	16 Oct – 23 Oct	M1: N-Tier setup, database schema, repositories, Identity roles, doctor listing
Implementation – Week 2	23 Oct – 30 Oct	M2: Booking calendar, conflict detection, doctor dashboard, SignalR

Phase	Dates	Milestone
Implementation – Week 3	30 Oct – 6 Nov	M3: Medical records, prescriptions, Twilio/SendGrid, patient profile
Implementation – Week 4	6 Nov – 13 Nov	M4: Admin dashboard, analytics, UI polish, Azure deployment
Stabilization & bug fixing	13 Nov – 30 Nov	M5: Feature freeze, full testing, documentation complete
Final presentation, testing & reports	30 Nov – 4 Dec 2026	M6: Final delivery and presentation

Gantt chart (weekly view). ■ = full activity, ▪ = light activity.

Task	2–16 Oct	16–23 Oct	23–30 Oct	30 Oct–6 Nov	6–13 Nov	13–30 Nov	30 Nov–4 Dec
Planning & documentation	■						
Architecture + DB + Auth + UI/UX foundation		■					
Appointments + Dashboard + SignalR			■				
Records + Prescriptions + Notifications				■			
Admin + Analytics + Azure					■		
Testing + bug fixing + docs					▪	■	■
Final presentation prep						▪	■

1.3 Task Assignment & Roles

Work is split evenly: each member owns 4 main features plus a set of smaller supporting tasks, and role assignments follow each member's stated preference (backend, frontend, UI/UX, or flexible).

Member	Role	Focus	Main responsibilities
Mohamed Munir	Team Leader + Backend / Integration	Technical coordination and integration	Planning, task tracking, meetings, final delivery. Solution setup (MediCare.Web / Services / Data, DI registration), conventions and branch/PR workflow. Twilio, SendGrid, 24-hour reminder scheduling and notification preferences. Admin dashboard. Azure deployment (App Service, SQL, SignalR Service), CI/CD, secrets. Removes blockers, reviews pull requests, merges releases
Ashraf Ibrahim	Backend Developer	Application services and booking logic	Appointment booking, conflict detection (including simultaneous requests), cancel/reschedule, status workflow, Factory pattern. Doctor working hours and vacations (services). Prescription services (medication, dosage, frequency, instructions). Analytics queries and PDF/Excel export

Member	Role	Focus	Main responsibilities
Natalie Bola	Backend Developer + Database	Data layer and backend integration	Entities and relationships (Doctors, Patients, Appointments, Specializations, WorkingHours, MedicalRecords), EF Core migrations and seed. Generic repository, specific repositories and Unit of Work with rollback. ASP.NET Identity, roles and registration validation. SignalR hub and real-time notifications. Medical records and file uploads with permissions
Malak Hany	Frontend Developer	MVC/Razor views	Doctor listing with search, filters and pagination. FullCalendar booking UI. Doctor dashboard (daily/weekly views, working hours, confirm/reject). Patient profile. Chart.js dashboards. Login and registration forms. SignalR notifications in the UI. Medical records and prescription views
Leqaa Ibrahim	UI/UX + Frontend Support + QA / Docs	User experience and clarity of flows	User flows and wireframes (Figma) for booking, doctor dashboard, prescription and admin. Bootstrap 5 design system, responsive layout, dark/light mode, accessibility, UI states (loading, empty, error, success). Prescription print layout. Test plan, booking/conflict test cases, bug tracking, test report. Compiles architecture doc, setup guide and user manual. Helps Malak build views when needed

All members also write documentation for their own modules and review at least one teammate's pull request per week. Every member builds their own features end to end; the focus areas only decide who helps first.

Feature ownership by week.

Wk	Feature	Owner	Support
1	Solution setup, N-Tier structure, DI, conventions	Mohamed Munir	Ashraf Ibrahim, Natalie Bola
1	Entities, relationships, EF Core migrations, seed	Natalie Bola	Ashraf Ibrahim
1	Repositories and Unit of Work	Natalie Bola	Mohamed Munir
1	Multi-role auth, Identity and registration	Natalie Bola	Mohamed Munir, Malak Hany (forms)
1	Doctor listing with search and filter	Malak Hany	Natalie Bola (queries), Leqaa Ibrahim (design)
1	UI/UX foundation: user flows, wireframes, design system	Leqaa Ibrahim	Malak Hany
2	Appointment booking, conflicts, reschedule, statuses, Factory	Ashraf Ibrahim	Natalie Bola
2	Doctor working hours and vacations (services)	Ashraf Ibrahim	Malak Hany
2	Booking calendar UI (FullCalendar) and doctor dashboard UI	Malak Hany	Ashraf Ibrahim, Leqaa Ibrahim
2	SignalR notifications	Natalie Bola	Ashraf Ibrahim (events), Malak Hany (UI)

Wk	Feature	Owner	Support
2	Booking and conflict-handling test cases	Leqaa Ibrahim	Ashraf Ibrahim
3	Medical records and uploads: services	Natalie Bola	Malak Hany
3	Medical records and prescription views	Malak Hany	Leqaa Ibrahim (use cases)
3	Prescriptions: services	Ashraf Ibrahim	Leqaa Ibrahim
3	Prescription print layout	Leqaa Ibrahim	Malak Hany
3	Twilio, SendGrid, reminders, notification preferences	Mohamed Munir	Natalie Bola
3	Patient profile	Malak Hany	Natalie Bola, Leqaa Ibrahim
4	Admin dashboard	Mohamed Munir	Malak Hany, Leqaa Ibrahim
4	Analytics: queries and PDF/Excel export	Ashraf Ibrahim	Mohamed Munir
4	Analytics: Chart.js dashboards	Malak Hany	Ashraf Ibrahim
4	UI polish: dark/light mode, accessibility, responsive checks	Leqaa Ibrahim	Malak Hany
4	Azure deployment and CI/CD	Mohamed Munir	Natalie Bola
4	Full testing, bug tracking, and test report	Leqaa Ibrahim	All members test their own modules
4	Documentation: architecture, setup guide, user manual	Leqaa Ibrahim	All members document their modules

1.4 Risk Assessment & Mitigation Plan

Probability (P) and Impact (I) are rated 1 (low) to 3 (high); Score = P × I.

#	Risk	P	I	Score	Mitigation	Contingency
R1	Team members have uneven experience with .NET	3	2	6	Pair programming, short knowledge-sharing sessions, code reviews	Reassign tasks by strength; leader helps on blockers
R2	Schedule slips because members are also students/employees	3	3	9	Realistic weekly targets, buffer weeks, early start, weekly tracking	Cut low-priority features (see MoSCoW)
R3	Merge conflicts and integration problems	2	2	4	Branching strategy, small PRs, frequent merges to develop	Leader-led integration session
R4	Double-booking bug or wrong availability logic	2	3	6	Unique constraints, transactions via Unit of Work, dedicated tests	Hotfix and add regression test
R5	Third-party service limits or failures (Twilio/SendGrid trials)	2	2	4	Early integration, abstraction behind interfaces, config switches	Mock/log-based notification fallback
R6	Azure cost, credit, or deployment problems	2	3	6	Deploy a skeleton early (Week 1–2), use free tiers, document setup	Deploy to alternative host or demo locally
R7	Security flaws exposing patient data	2	3	6	Identity roles/policies, authorization attributes, HTTPS, input validation, secrets in config/Key Vault	Security review before release
R8	Scope creep	3	2	6	Fixed scope baseline, change requests approved by leader	Move extras to backlog

#	Risk	P	I	Score	Mitigation	Contingency
R9	A team member becomes unavailable	2	3	6	Shared code ownership, documentation, backup owner per module (the "Support" column in 1.3)	Redistribute tasks at once
R10	Data loss or lost source code	1	3	3	GitHub repo, regular database scripts/migrations, backups	Restore from repo and migrations
R11	Insufficient testing time	2	3	6	Tests written alongside features, QA owner, stabilization window	Prioritize critical-path testing
R12	Poor communication or unclear requirements	2	2	4	Weekly meetings, written decisions in repo, shared board	Clarify with instructor early

Review. Risks are reviewed in each weekly meeting; scores and statuses are updated in the repository.

Stakeholder conflicts and risks (from the stakeholder analysis). Score = Likelihood × Impact (1–5 each). Priority: High ≥ 15, Medium 8–14, Low < 8.

#	Conflict / risk	Description	Stakeholders	Mitigation	L	I	Score	Priority
S1	Doctors vs. patients on availability	Patients want maximum flexibility; doctors want controlled schedules and vacation blocks	Doctors, Patients	Conflict detection and availability management features	3	3	9	Medium
S2	Privacy vs. convenience	Medical records and SMS/email notifications carry privacy risks	Patients, Regulators / legal	Keep sensitive details out of messages; customizable notification preferences	3	5	15	High
S3	Scope vs. time	Booking, records, prescriptions, notifications, analytics and Azure deployment in 4 weeks	Dev team, Supervisor / client	Prioritize core flows (booking, doctor dashboard); treat analytics and dark mode as lower priority	4	4	16	High
S4	Third-party dependency	Twilio, SendGrid or Azure SignalR failures or quota limits break reminders	Vendors, Patients, Doctors	Fallbacks and error logging; monitor quotas	3	3	9	Medium
S5	Doctor approval workflow	Admin must approve doctor registrations, which can delay onboarding	Clinic admin, Doctors	Clear approval SLA; pending-status notifications to doctors	3	2	6	Low
S6	Missing role	The spec has only Patient, Doctor and Admin; real clinics also have reception staff	Reception staff, Clinic admin	Consider adding a Receptionist role	3	2	6	Low

1.5 KPIs (Key Performance Indicators)

The team defined ten product KPIs. Targets marked as initial proposals are confirmed after a baseline is measured. Each KPI needs its terms defined consistently before measuring (what counts as an attempt, a success, the reporting period).

#	KPI	Formula	Target	Data required and notes
1	Successful Booking Rate	successful booking attempts ÷ total booking attempts × 100	≥ 95% (initial proposal)	Booking attempts and outcomes. Define "attempt" and "success"; success = appointment saved correctly in the database
2	Average Booking Completion Time	total booking duration ÷ completed bookings	< 2 minutes (initial proposal)	Start and completion time of each booking. Start = patient begins selecting a slot; end = booking submitted
3	Appointment Cancellation Rate	cancelled appointments ÷ confirmed appointments in scope × 100	No fixed target; set a baseline first	Statuses, confirmation, cancellation and appointment dates. A high rate is not necessarily a system problem; fix the reporting period and denominator
4	Double Booking Rate	conflicted confirmed appointments ÷ total confirmed appointments × 100	0%	Appointment IDs, doctor IDs, start/end times, statuses, conflict- check results. Test simultaneous bookings of the same slot, not only sequential ones; prevent conflicts at confirmation
5	Notification Delivery Rate	successfully delivered notifications ÷ notifications sent × 100	≥ 98% (initial proposal, subject to testing and provider capabilities)	Notification IDs, delivery statuses, timestamps, channels, provider delivery reports. "Sent" is not "delivered"; measure SignalR, SMS (Twilio) and email (SendGrid) separately
6	Average Appointments per Doctor	total booked appointments ÷ total registered doctors	≥ 5 per doctor per month*	Booked appointments in the period; registered doctors. Fix the period; a higher average is not always better (different specialties and hours)
7	Doctor Availability Rate	doctors with available slots ÷ registered doctors × 100	≥ 80%*	Doctors with at least one bookable slot in a defined period (e.g. next 7 days); does not show whether slots meet demand
8	Medical Records Completeness Rate	complete medical records ÷ total medical records × 100	100%	Record fields, required-field definitions, validation results. Required: e.g. diagnosis, symptoms, treatment notes; optional documents (X-rays, results) do not make a record incomplete; measures completeness, not medical correctness

#	KPI	Formula	Target	Data required and notes
9	Report Data Accuracy	report values compared with independently verified results (same definitions and filters)	100% agreement on tested report metrics and scenarios	Database records, report outputs, agreed calculation rules, reference results. Test date ranges, statuses and empty datasets; one matching case does not prove all reports are accurate
10	System Availability (Uptime)	available time ÷ total monitored time × 100	≥ 99.5% (initial proposal, subject to hosting and monitoring)	Monitoring results, timestamps, availability-check outcomes, downtime records. Needs a defined period and a clear definition of "available"; a basic check does not prove every feature works

* The source KPI file listed these two targets the other way round; they are matched here to each KPI's formula (an average count vs. a percentage).

Project delivery and quality KPIs

Category	KPI	Target	How measured
Performance	Page response time (main pages)	< 2 seconds under normal load	Browser dev tools, Application Insights
Quality	Test pass rate	≥ 90%; 100% for critical flows	Test report
Quality	Critical/high-severity bugs open at release	0	GitHub Issues
Delivery	Milestones on time	≥ 80%	Project board
Delivery	Planned features delivered	100% of Must-have, ≥ 80% of Should- have	Requirements checklist
Adoption	Pilot user satisfaction; usability task success	≥ 4 / 5; ≥ 90%	Short survey (10+ testers), usability test
Team	Pull requests reviewed within 24 hours	≥ 80%	GitHub
Team	Weekly meeting attendance	≥ 90%	Meeting notes

2. Literature Review

This section is completed with the lecturer and is mostly filled after review sessions. Below is the structure and a starting draft.

2.1 Feedback & Evaluation – Lecturer's Assessment

Overall evaluation: [to be written after the final assessment]

Date	Reviewer	Area reviewed	Feedback	Status
[date]	[Lecturer name]	Proposal and scope	[to be filled]	[Open / Done]
[date]	[Lecturer name]	Architecture and database design	[to be filled]	[Open / Done]
[date]	[Lecturer name]	Mid-project demo	[to be filled]	[Open / Done]
[date]	[Lecturer name]	Final presentation	[to be filled]	[Open / Done]

2.2 Suggested Improvements

Self-identified improvements for future versions, to be extended with lecturer suggestions: online payment integration for consultation fees; video consultation (telemedicine); multi-clinic and multi-branch support; Arabic/English localization with RTL support; mobile app or Progressive Web App; doctor ratings and patient reviews; waiting- list and automatic rescheduling when a slot opens; advanced audit analytics and stronger data-privacy controls for medical data; automated test coverage and performance testing; insurance and billing module.

2.3 Final Grading Criteria

Exact marks are set by the lecturer; the table below is a proposed breakdown to confirm with them.

Component	Weight	What is assessed
Documentation	20%	Planning, requirements, architecture, user manual, clarity and completeness
Implementation	40%	Working features, N-Tier design, patterns (Repository, Unit of Work, Factory, DI), code quality
Testing	20%	Test plan, test cases, conflict handling, bug reports, results
Presentation	20%	Live demo, teamwork, answering questions, time management

3. Requirements Gathering

3.1 Stakeholder Analysis

Power and Interest are scored from 1 (lowest) to 5 (highest); a score of 3 or more is rated High.

#	Stakeholder	Type	Main interests / expectations	Concerns / risks	Power	Interest	Strategy
1	Patients	External, primary user	Easy doctor search, quick booking, reminders, access to records and prescriptions	Privacy of medical data, confusing UI, missed notifications	2	5	Keep informed
2	Doctors	External/internal, primary user	Manage schedule, confirm or reject bookings, write prescriptions, see patient history quickly	Time- consuming workflows, double bookings, system downtime	5	5	Manage closely
3	Clinic admin / management	Internal, primary user	Full control over doctors, patients, specializations, appointments and reports	Complex dashboard, inaccurate analytics, lack of control	5	5	Manage closely
4	Clinic owner / project sponsor	Internal	ROI, revenue reports, more bookings, fewer no- shows, scalability	Cost overruns, delays, low adoption	5	5	Manage closely
5	Development team	Internal	Clear requirements, realistic timeline, clean N-Tier architecture	Scope creep, technical debt, tight 4-week schedule	4	5	Manage closely
6	Project supervisor / instructor / client	Internal/external	Weekly deliverables, correct use of design patterns, documentation quality	Missed milestones, poor code quality	4	4	Manage closely
7	Reception / support staff	Internal, secondary user	Help patients book, reschedule and cancel; see the day's appointments	Not covered by a dedicated role; may be forced to use the Admin role	2	3	Keep informed
8	Microsoft Azure (App Service, SQL, SignalR)	External vendor	Correct and paid usage of services	Cost, outages, configuration limits	3	2	Keep satisfied
9	Twilio and SendGrid	External vendor	API usage within terms and quotas	Delivery failures, cost, spam flagging, rate limits	3	2	Keep satisfied

#	Stakeholder	Type	Main interests / expectations	Concerns / risks	Power	Interest	Strategy
10	Regulators / legal authorities (HIPAA, GDPR, local health-data laws)	External	Protection and lawful handling of medical data, consent, audit trails	Fines, legal liability, data breaches	5	2	Keep satisfied
11	Labs / radiology centers	External, indirect	Their test results and X- rays are uploaded to patient records	File format compatibility, data integrity	2	3	Keep informed
12	Pharmacies	External, indirect	Readable, authentic printed prescriptions	Forged or unclear prescriptions	1	2	Monitor
13	Insurance companies	External, indirect	Accurate billing and visit records	Data mismatches, fraud	1	2	Monitor
14	DevOps / maintenance team	Internal	Stable CI/CD, monitoring, backups	Failed deployments, unclear documentation	2	3	Keep informed
15	Competitors / existing clinic systems	External	Market share	Competitive features and pricing	1	1	Monitor

Patients carry low individual power but are the largest user group, so adoption and usability testing remain a priority.

Power / interest grid. Count by strategy: Manage closely 5 · Keep satisfied 3 · Keep informed 4 · Monitor 3 · Total 15.

	Low interest	High interest
High power	Keep satisfied: Microsoft Azure, Twilio and SendGrid, Regulators / legal authorities	Manage closely: Doctors, Clinic admin / management, Clinic owner / sponsor, Development team, Supervisor / client
Low power	Monitor: Pharmacies, Insurance companies, Competitors / existing clinic systems	Keep informed: Patients, Reception / support staff, Labs / radiology centers, DevOps / maintenance team

Engagement strategy

Stakeholder group	Strategy	How	When	Responsible
Doctors	Involve in design and testing	Interviews, UI prototype review, UAT sessions	Weeks 1–3	Project manager (Mohamed Munir)
Clinic admin and owner	Regular updates and sign-off	Weekly demos, reports review, approval of the admin dashboard	Every week, final in Week 4	Project manager (Mohamed Munir)
Patients	Gather feedback, ensure usability	Short surveys, usability testing of booking and profile pages	Weeks 2–3	UX lead (Leqaa Ibrahim)
Dev team	Align internally	Daily or weekly stand-ups, task board, code reviews	Throughout	Development lead (Ashraf Ibrahim)
Supervisor / client	Transparent progress reporting	Deliverable demos at the end of each week	Weekly	Project manager (Mohamed Munir)
Regulators / legal	Compliance by design	Encryption, role- based access, audit logs, consent, privacy policy	Design in Week 1, verify in Week 4	Technical lead (Mohamed Munir)
Vendors (Azure, Twilio, SendGrid)	Monitor and manage	Track quotas, costs and service status	Weeks 3–4	DevOps engineer (Mohamed Munir)
Reception staff	Inform and train	Short guide and training, or consider adding a Receptionist role	Week 4	Project manager (Mohamed Munir)

3.2 User Stories & Use Cases

Patient user stories

US-P1: As a patient, I want to register and log in so that I can use the system securely.

US-P2: As a patient, I want to search doctors by specialization, location, and availability so that I can find the right one.

US-P3: As a patient, I want to see a doctor's available time slots on a calendar so that I can pick a suitable time.

US-P4: As a patient, I want to book, cancel, or reschedule an appointment so that I can manage my visits.

US-P5: As a patient, I want SMS, email, and in-app reminders so that I do not miss appointments.

US-P6: As a patient, I want to view my appointment history, medical records, and prescriptions so that I can follow my treatment.

US-P7: As a patient, I want to update my personal information, allergies, and medical history so that my data is accurate.

US-P8: As a patient, I want to download my medical documents and prescriptions so that I can keep copies.

Doctor user stories

US-D1: As a doctor, I want to register and wait for admin approval so that only verified doctors are listed.

US-D2: As a doctor, I want to set working hours and vacation days so that patients only book valid slots.

US-D3: As a doctor, I want to view daily and weekly appointments in calendar and list views so that I can plan my day.

US-D4: As a doctor, I want to confirm or reject appointment requests so that I control my schedule.

US-D5: As a doctor, I want to open a patient's information and history quickly so that I can treat them better.

US-D6: As a doctor, I want to record diagnosis, symptoms, treatment notes, and upload test results so that the visit is documented.

US-D7: As a doctor, I want to create and print digital prescriptions so that patients get clear instructions.

US-D8: As a doctor, I want instant notification of new bookings so that I can respond quickly.

Admin user stories

US-A1: As an admin, I want to approve or update doctor profiles so that the directory stays trustworthy.

US-A2: As an admin, I want to manage patients and their access so that I can handle problems.

US-A3: As an admin, I want to manage specializations and clinic settings so that the system matches the clinic.

US-A4: As an admin, I want to view and manage all appointments so that I can resolve conflicts.

US-A5: As an admin, I want charts and reports on appointments, popular doctors, demographics, and revenue so that I can make decisions.

US-A6: As an admin, I want to export reports to PDF and Excel so that I can share them.

Key use cases

Use case	Actor	Main flow	Alternative / exception flows
UC1: Register account	Patient / Doctor	Choose role, fill form, validate, create account, confirm email	Duplicate email; invalid data; doctor stays pending until approval
UC2: Search doctors	Patient	Enter specialization/location/availability, view filtered list, open profile	No results found
UC3: Book appointment	Patient	Select doctor, choose slot in calendar, confirm, system checks conflict, saves as Pending, notifies doctor	Slot already taken; doctor on vacation; patient has overlapping appointment
UC4: Confirm/reject appointment	Doctor	Open request, confirm or reject with reason, patient notified	Request already cancelled
UC5: Cancel/reschedule	Patient	Open appointment, cancel or choose new slot, system rechecks availability, notifies doctor	Cancellation too close to appointment time (if policy applies)
UC6: Create medical record	Doctor	Open visit, enter diagnosis/symptoms/notes, upload files, save	Upload fails or file type not allowed
UC7: Issue prescription	Doctor	Add medications with dosage and frequency, save, print or share	Missing required fields
UC8: Send reminders	System	Scheduler finds appointments in 24 hours, sends SMS and email per preferences	Provider failure leads to retry and logging

Use case	Actor	Main flow	Alternative / exception flows
UC9: Manage doctors	Admin	Review pending doctors, approve or reject, edit profile	Rejected doctor notified
UC10: View analytics	Admin	Open dashboard, choose period, view charts, export	No data for the period

3.3 Functional Requirements

Week 1 – Initial Setup and User Management

Scope. N-Tier solution setup, database design, repository and Unit of Work layer, multi- role authentication with role-specific registration, and the doctor listing page with search and filters.

Role	Description in Week 1
Patient	Registers an account, signs in, and browses the doctor listing
Doctor	Registers with professional details, signs in, and appears in the doctor listing
Admin	Pre-created account that signs in with full administrative role access
Visitor	Unauthenticated user who can view the doctor listing and the registration and login pages

N-Tier architecture

ID	Requirement	Priority
FR-ARC-01	The solution shall contain three projects: MediCare.Web (presentation), MediCare.Services (business logic), and MediCare.Data (data access)	Must
FR-ARC-02	MediCare.Web shall contain the MVC controllers, views, and client-side logic, and shall not access the database directly	Must
FR-ARC-03	MediCare.Services shall contain the business rules and shall be the only layer called by controllers	Must
FR-ARC-04	MediCare.Data shall contain the entities, the DbContext, the repositories, and the Unit of Work	Must
FR-ARC-05	Project references shall flow in one direction only: Web to Services to Data	Must
FR-ARC-06	Services and repositories shall be registered in the dependency injection container and injected into controllers and services through their interfaces	Must

Database (schema defined with Entity Framework Core and created through migrations)

ID	Requirement	Priority
FR-DB-01	The system shall store doctors with personal information, specialization, consultation fee, profile image, and location	Must
FR-DB-02	The system shall store patients with personal information, medical history, and allergies	Must
FR-DB-03	The system shall store appointments with details, status, and notes, linked to one doctor and one patient	Must
FR-DB-04	The system shall store medical specializations as a separate table, and each doctor shall belong to one specialization	Must
FR-DB-05	The system shall store working hours (doctor availability schedules) linked to a doctor, with day of week, start time, and end time	Must
FR-DB-06	The system shall store medical records (patient medical records and history) linked to a patient and a doctor	Must

ID	Requirement	Priority
FR-DB-07	Relationships, required fields, maximum lengths, and foreign key constraints shall be defined for every table	Must
FR-DB-08	The database shall be created and updated through EF Core migrations, and a seed routine shall load the three roles, the initial Admin account, and a starter list of specializations	Must

Repository and Unit of Work

ID	Requirement	Priority
FR-REP-01	A generic repository shall provide create, read (by ID and list), update, and delete operations for all entities	Must
FR-REP-02	IDoctorRepository shall extend the generic repository with queries for doctors by specialization, location, and availability	Must
FR-REP-03	IPatientRepository shall extend the generic repository with patient-specific queries	Must
FR-REP-04	IAppointmentRepository shall extend the generic repository with appointment queries by doctor, patient, and date	Must
FR-REP-05	IMedicalRecordRepository shall extend the generic repository with queries by patient	Must
FR-REP-06	A Unit of Work shall expose all repositories and a single save operation that commits related changes in one transaction	Must
FR-REP-07	If any part of a grouped operation fails, the Unit of Work shall roll back the entire transaction	Must

Authentication and registration (ASP.NET Identity with roles Patient, Doctor, Admin)

ID	Requirement	Priority
FR-AUTH-01	The system shall define the roles Patient, Doctor, and Admin and assign every user exactly one role	Must
FR-AUTH-02	The system shall provide a patient registration form collecting full name, email, phone number, date of birth, gender, and password, with optional medical history and allergies	Must
FR-AUTH-03	The system shall provide a doctor registration form collecting full name, email, phone number, password, specialization, consultation fee, location, and profile image	Must
FR-AUTH-04	Admin accounts shall not be self-registered; they shall be created by the seed routine	Must
FR-AUTH-05	Registration shall validate required fields, email format, unique email, phone number format, and password strength (minimum length, upper and lower case, digit, and symbol)	Must
FR-AUTH-06	Doctor registration shall also validate that the consultation fee is a positive number and that the profile image is a permitted type (JPG or PNG) within the size limit	Must
FR-AUTH-07	Registration shall create the Identity user, assign the correct role, and create the matching Patient or Doctor profile in a single transaction	Must

ID	Requirement	Priority
FR-AUTH-08	The system shall let registered users sign in with email and password and sign out	Must
FR-AUTH-09	After sign-in, the system shall redirect the user to a landing page appropriate to their role	Must
FR-AUTH-10	The system shall restrict controllers and actions by role, redirect unauthenticated users to the login page, and unauthorized users to an access-denied page	Must
FR-AUTH-11	The system shall lock an account temporarily after repeated failed sign-in attempts	Should
FR-AUTH-12	Passwords shall be stored hashed through ASP.NET Identity and shall never be displayed or logged	Must

Doctor listing

ID	Requirement	Priority
FR-DOC-01	The system shall provide a doctor listing page showing each doctor's name, photo, specialization, consultation fee, and location	Must
FR-DOC-02	The doctor listing shall be visible to visitors, patients, doctors, and admins	Must
FR-DOC-03	The user shall be able to search doctors by name	Must
FR-DOC-04	The user shall be able to filter doctors by specialization, chosen from the specializations table	Must
FR-DOC-05	The user shall be able to filter doctors by location	Must
FR-DOC-06	The user shall be able to filter doctors by availability, based on the doctors' working hours (for example, by day of week)	Must
FR-DOC-07	Search and filters shall be combinable, and results shall show only doctors matching all selected criteria	Must
FR-DOC-08	The user shall be able to clear all filters and return to the full list	Should
FR-DOC-09	When no doctor matches, the page shall show a clear "no results" message	Must
FR-DOC-10	The listing shall be paginated when the number of doctors exceeds one page	Should
FR-DOC-11	Each listing entry shall link to a doctor details page	Should

Out of scope for Week 1: appointment booking, calendar, cancellation, rescheduling and conflict detection; doctor dashboard and working-hours management through the interface; admin approval of doctor registrations; notifications (real-time, SMS, email); medical record entry, file uploads and prescriptions; patient profile page, admin dashboard, analytics and Azure deployment. The appointments, working hours and medical records tables are created in Week 1 as database structure only.

Week 1 deliverable	Requirements
Full N-Tier architecture with separation of Presentation, Business Logic, and Data Access	FR-ARC-01 to FR-ARC-06
Multi-role authentication and registration system	FR-AUTH-01 to FR-AUTH-12
Database schema in EF Core with repositories and Unit of Work	FR-DB-01 to FR-DB-08, FR-REP-01 to FR- REP-07
Doctor listing page with search and filter by specialization, location, and availability	FR-DOC-01 to FR-DOC-11

Week 2 – Appointment System and Doctor Dashboard

ID	Requirement	Priority
FR-2.01	The system shall show each doctor's available time slots in an interactive calendar (FullCalendar.js)	Must
FR-2.02	Patients shall book an appointment by selecting a date and time based on the doctor's availability	Must
FR-2.03	Patients shall cancel and reschedule their appointments	Must
FR-2.04	The system shall detect conflicts and prevent double bookings, including simultaneous requests for the same slot	Must
FR-2.05	Appointments shall have the statuses Pending, Confirmed, Rejected, Cancelled, and Completed	Must
FR-2.06	A Factory pattern shall create the different types of appointments and notifications	Must
FR-2.07	Doctors shall view daily and weekly appointments in calendar and list views	Must
FR-2.08	Doctors shall manage their working hours and vacation days	Must
FR-2.09	Doctors shall confirm or reject appointment requests	Must
FR-2.10	Doctors shall quickly access patient information from an appointment	Should
FR-2.11	The system shall notify the doctor instantly (SignalR) when a new appointment is booked	Must
FR-2.12	The system shall notify the patient instantly (SignalR) when an appointment is confirmed or rejected	Must
FR-2.13	The system shall send reminders before scheduled appointments	Should
FR-2.14	The booking system and conflict handling shall be tested comprehensively	Must

Week 3 – Medical Records, Prescriptions, External Notifications and Patient Profile

Medical records management

ID	Requirement
FR-3.1	The system shall allow an authorized doctor to create a medical record for a patient visit
FR-3.2	The system shall allow the doctor to record the patient's diagnosis, symptoms, and treatment notes for each visit
FR-3.3	The system shall allow authorized users to upload and attach medical documents, including test results, X-rays, and other relevant files
FR-3.4	The system shall allow authorized users to view a patient's medical history and previous visit records
FR-3.5	The system shall associate each medical record with the relevant patient and visit
FR-3.6	The system shall restrict access to medical records and attached documents according to user roles and permissions

Prescription system

ID	Requirement
FR-3.7	The system shall allow an authorized doctor to create a digital prescription for a patient
FR-3.8	The system shall allow the doctor to add medication details, dosage, frequency, and usage instructions to a prescription

ID	Requirement
FR-3.9	The system shall save each prescription and associate it with the relevant patient and visit
FR-3.10	The system shall allow patients to view their prescription history
FR-3.11	The system shall provide a print-friendly prescription format and allow the prescription to be printed

External notifications

ID	Requirement
FR-3.12	The system shall send SMS appointment reminders through Twilio when the integration is configured
FR-3.13	The system shall send email notifications through SendGrid when the integration is configured
FR-3.14	The system shall schedule an automated reminder 24 hours before a scheduled appointment
FR-3.15	The system shall allow users to manage their notification preferences
FR-3.16	The system shall notify patients when an appointment is confirmed or rejected, using the notification channels enabled for that patient

Patient profile

ID	Requirement
FR-3.17	The system shall allow patients to view their upcoming appointments and appointment history
FR-3.18	The system shall allow patients to access their medical records and prescriptions, subject to authorization rules
FR-3.19	The system shall allow patients to update their personal information and medical history
FR-3.20	The system shall allow patients to download medical documents that they are authorized to access

Week 4 – Admin Dashboard, Analytics and Deployment

ID	Requirement	Priority
FR-4.01	Admins shall approve doctor registrations and update doctor profiles	Must
FR-4.02	Admins shall manage patients and their access	Must
FR-4.03	Admins shall manage specializations and clinic settings	Must
FR-4.04	Admins shall view and manage all appointments	Must
FR-4.05	The system shall display monthly appointment statistics (Chart.js)	Must
FR-4.06	The system shall display the most popular doctors and specializations	Must
FR-4.07	The system shall display patient demographics and trends	Must
FR-4.08	The system shall provide revenue reports and financial summaries, where revenue = the consultation fee of each Completed appointment (no online payments in this release)	Must
FR-4.09	Reports shall be exportable to PDF and Excel	Should
FR-4.10	The UI shall be responsive (Bootstrap 5) with intuitive navigation	Must
FR-4.11	The UI shall include accessibility improvements	Must
FR-4.12	The UI shall offer a dark/light mode toggle	Could

ID	Requirement	Priority
FR-4.13	The application shall be deployed to Azure App Service with Azure SQL Database	Must
FR-4.14	Real-time features shall run on Azure SignalR Service	Must
FR-4.15	A CI/CD pipeline shall be set up with Azure DevOps or GitHub Actions	Should
FR-4.16	The project shall include documentation: architecture explanation with diagrams, API documentation, setup guide, and user manual	Must

3.4 Non-Functional Requirements

Category	Requirements
Performance	NFR-1: Main pages load in under 2 seconds under normal load (about 50 concurrent users). NFR-2: Appointment conflict check completes in under 1 second. NFR-3: Real-time notifications arrive within 3 seconds.
Security and privacy	NFR-4: Passwords are hashed using ASP.NET Identity; HTTPS is enforced. NFR-5: Role-based and resource-based authorization: patients only see their own data; doctors only see their patients. NFR-6: Protection against SQL injection (EF Core parameterization), XSS, and CSRF (anti-forgery tokens). NFR-7: Uploaded files are validated by type and size and stored securely. NFR-8: Secrets (API keys, connection strings) are stored in configuration/Key Vault, never in source control. Medical records, prescriptions and attached documents are visible only to authorized users; one patient's information is never exposed to another patient. Accounts lock temporarily after repeated failed sign-in attempts; passwords are never displayed or logged. Compliance by design with health-data regulations (HIPAA, GDPR, local laws): encryption, role-based access, a basic audit log of key actions (sign-in, record access, appointment status changes), consent, and a privacy policy; sensitive details are kept out of SMS and email messages. Upload file types and size limits, and notification retry behavior, are agreed during requirements gathering.
Usability and accessibility	NFR-9: A new patient can book an appointment in under 2 minutes without training. NFR-10: The UI is responsive on mobile, tablet, and desktop, and follows basic accessibility guidelines (labels, contrast, keyboard navigation). NFR-11: Clear error messages and form validation.
Reliability and availability	NFR-12: Target uptime of at least 99.5% during the evaluation period. NFR-13: Related changes are saved in a single transaction (Unit of Work) to avoid inconsistent data. NFR-14: Notification failures are logged and retried without breaking the booking flow.
Maintainability and scalability	NFR-15: Strict N-Tier separation (Web / Services / Data) with Repository, Unit of Work, and Dependency Injection. NFR-16: Code follows consistent naming conventions and is reviewed through pull requests. NFR-17: Business logic is unit-testable through interfaces. NFR-18: The design allows adding new notification channels or appointment types without major changes. NFR- 22: Code follows SOLID principles and public APIs carry XML documentation. NFR-23: Business logic is covered by unit tests and the repository layer by integration tests; critical paths (sign-in, booking, prescription) get basic UI tests. NFR-24: Errors are handled consistently and logged centrally (level, user/request context), with no sensitive data in logs.
Compatibility and portability	NFR-19: Works on current versions of Chrome, Edge, Firefox, and Safari. NFR-20: Database changes are managed through EF Core migrations.
Documentation	NFR-21: Architecture explanation with diagrams, API documentation, setup guide, and user manual are provided in the repository.